Docker Technical Note — User Management API

Stack: Node.js 20 | Express | Prisma ORM | PostgreSQL 18 | Docker Compose Date: March 2026

1 CONTAINER ARCHITECTURE

Two containers are orchestrated by Docker Compose. The API container never touches the host database directly — all DB traffic stays on the internal Docker network.

🖥️ HOST MACHINE

- Port **5501** → API
- Port **5432** → PostgreSQL

⇄

🗄 CONTAINER: NODEAPI

- Image: `napiserver` (built once)
- Internal port: 5501
- Env vars from `.env`
- Migrations auto-run on startup

🗄 CONTAINER: POSTGRESDB

- Image: `postgres:18`
- Internal port: 5432
- DB name set per environment
- Data persisted via Docker volume

⇄

💾 DOCKER VOLUME

- `postgres_data`
- Survives `compose down`
- prod uses `postgres_prod_data`

> **Internal networking:** The API container reaches PostgreSQL via the service name `postgresDB` (Docker bridge network), not via `localhost`. The `DATABASE_URL` in each `.env` file uses this service name.

2 HOW ENVIRONMENTS ARE MAINTAINED

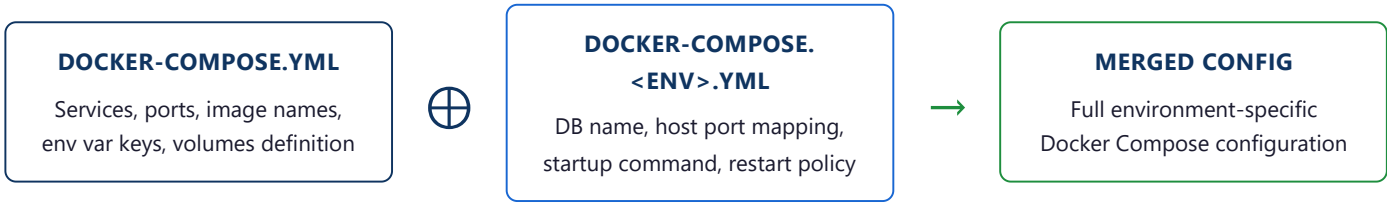
All environment-specific settings live in **env_templates/**, committed to the repo. Docker Compose reads variables from a single `.env` file at runtime. Switching envs means replacing that one file — no code changes, no image rebuild.

Template file	Environment	DB name	Compose override	Node behaviour
`env_templates/.env.dev`	Development	`devdb`	`docker-compose.dev.yml`	Hot-reload (`nodemon`), `migrate dev`
`env_templates/.env.test`	Test	`testdb`	`docker-compose.test.yml`	Run tests, `--abort-on-container-exit`
`env_templates/.env.demo`	Demo	`demodb`	`docker-compose.demo.yml`	Production-mode server, `migrate deploy`
`env_templates/.env.prod`	Production	`proddb`	`docker-compose.prod.yml`	`restart: always` , persistent volume

> **Why only one env at a time?** All environments expose the API on host port `5501` . The OS can only bind one process to a given port, so two environments cannot run simultaneously on the same machine.

3 COMPOSE FILE LAYERING

Docker Compose merges a **base file** with an **environment override**. The override only redefines what differs (DB name, host port, startup command). Everything else — image name, internal networking, volume mounts — comes from the base.



4 SWITCHING ENVIRONMENTS

Three steps, always the same pattern. No code changes, no rebuild.

1. **Stop current environment**

   ```
   docker compose down
   ```

2. **Swap the .env file**

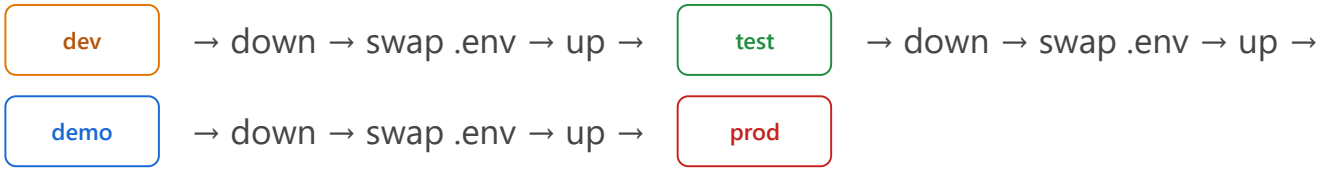
   ```
   copy env_templates\.env.<env> .env
   ```

   Overwrites the previous `.env` — this is the *only* change between environments

3. **Start the next environment**

   ```
   docker compose -f docker-compose.yml -f docker-compose.<env>.yml up
   ```

   Migrations run automatically inside the container on startup



5 FIRST TIME SETUP (FRESH CLONE)

The Docker image (`napiserver`) is built **once** and reused across all environments. After cloning, you can go straight to **any** environment — you do not have to start with dev.

```
# 1. Be in the postgres/ directory
cd node/database/postgres

# 2. Pick any environment — e.g. demo
copy env_templates\.env.demo .env

# 3. Build the image once (only needed on first run)
docker compose build --no-cache

# 4. Start — migrations run automatically inside the container
docker compose -f docker-compose.yml -f docker-compose.demo.yml up
```

> **Rebuild?** Only needed when the `Dockerfile` , `package.json` , or application source changes. Swapping environments *never* requires a rebuild.

6 QUICK REFERENCE

Action	Command
Build image	`docker compose build --no-cache`
Start dev	`docker compose -f docker-compose.yml -f docker-compose.dev.yml up`
Start test	`docker compose -f docker-compose.yml -f docker-compose.test.yml up --abort-on-container-exit`
Start demo	`docker compose -f docker-compose.yml -f docker-compose.demo.yml up`
Start prod	`docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d`
Stop any env	`docker compose down`
View container logs	`docker compose logs`
Check running containers	`docker compose ps`
Reset (nuke & rebuild)	`docker compose down && docker compose build --no-cache`